Latent-KalmanNet: Learned Kalman Filtering for Tracking From High-Dimensional Signals

Itay Buchnik, *Student Member, IEEE*, Guy Revach, *Graduate Student Member, IEEE*, Damiano Steger, *Student Member, IEEE*, Ruud J. G. van Sloun, *Member, IEEE*, Tirza Routtenberg, *Senior Member, IEEE*, and Nir Shlezinger, *Senior Member, IEEE*

***Abstract*—The Kalman filter (KF) is a widely used algorithm for tracking dynamic systems that are captured by state space (SS) models. The need to fully describe an SS model limits its applicability under complex settings, e.g., when tracking based on visual data, and the processing of high-dimensional signals often induces notable latency. These challenges can be treated by mapping the measurements into latent features obeying some postulated closed-form SS model, and applying the KF in the latent space. However, the validity of this approximated SS model may constitute a limiting factor. In this work, we study tracking from high-dimensional measurements under complex settings using a hybrid model-based/data-driven approach. By gradually tackling the challenges in handling the measurement model and the task, we develop Latent-KalmanNet, which implements tracking from high-dimensional measurements by leveraging data to jointly learn the KF along with the latent space mapping. Latent-KalmanNet combines a learned encoder with data-driven tracking in the latent space using the recently proposed-KalmanNet, while identifying the ability of each of these trainable modules to assist its counterpart via providing a suitable prior (by KalmanNet) and by learning a latent representation that facilitates data-aided tracking (by the encoder). Our empirical results demonstrate that the proposed Latent-KalmanNet achieves improved accuracy and run-time performance over both model-based and data-driven techniques by learning a surrogate latent representation that most facilitates tracking, while operating with limited complexity and latency.**

Manuscript received 14 April 2023; revised 21 September 2023 and 20 November 2023; accepted 1 December 2023. Date of publication 22 December 2023; date of current version 4 January 2024. This work was supported in part by the Israeli Ministry of National Infrastructure, Energy, and Water Resources. An earlier version of this paper was presented in part at the 2023 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP) [DOI: 10.1109/ICASSP49357.2023.10096006]. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Mojtaba Soltanalian. *(Corresponding author: Itay Buchnik.)*

Itay Buchnik and Nir Shlezinger are with the School of Electrical and Computer Engineering, Ben-Gurion University of the Negev, Be'er Sheva 84105, Israel (e-mail: itaybuch@post.bgu.ac.il; tirzar@bgu.ac.il; nirshl@bgu.ac.il).

Guy Revach and Damiano Steger are with the Department of Information Technology and Electrical Engineering (D-ITET), The Institute for Signal and Information Processing (ISI), ETH Zürich, 8092 Zürich, Switzerland (e-mail: grevach@ethz.ch; stegerd@student.ethz.ch).

Ruud J. G. van Sloun is with the Electrical Engineering Department, Eindhoven University of Technology, 5612 AZ Eindhoven, The Netherlands (e-mail: r.j.g.v.sloun@tue.nl).

Tirza Routtenberg is with the School of Electrical and Computer Engineering, Ben-Gurion University of the Negev, Be'er Sheva, Israel and also with the Electrical and Computer Engineering Department, Princeton University, Princeton, NJ 08544 USA.

Digital Object Identifier 10.1109/TSP.2023.3344360

***Index Terms*—Data-aided tracking, high-dimensional measurements, Kalman filters, latent space, model-based deep-learning, nonlinear state-space models, model-based deep-learning.**

I. Introduction

TRACKING the hidden state of dynamic systems is a fundamental problem in various fields, including signal processing, control, and finance. In many real-world applications, such as autonomous driving, smart city monitoring, and visual surveillance, tracking is based on noisy high-dimensional observations, e.g., visual data. The classic KF [2] algorithm and its variants have been the go-to approach for tracking, relying on the representation of the dynamics as a SS model that describes the state evolution and the measurement model [3, Ch. 10]. KF is widely used due to its computational efficiency and optimality properties. However, the reliance of the KF and its variants on an accurate description of the underlying dynamics as a closed-form SS model with Gaussian noise restricts it applicability when tracking from complex high-dimensional data.

In particular, the KF assumes linear dynamics with Gaussian noise of a known distribution. Variations of the KF, such as the extended (EKF) [4] and the unscented Kalman filter [5], can cope with nonlinear Gaussian SS models, yet they require an accurate description of the nonlinearities, which is often unavailable when dealing with visual data. Alternative tracking methods are based on nonlinear/non-Gaussian particle filters, which can be applied directly to the signal [6], [7] or to linear projections as in [8]. However, these become computationally complex when dealing with high-dimensional signals, and linear projections may struggle in dealing with complex data, e.g., images. While for certain families of high-dimensional observations, such as graph signals, one can leverage structures in the data to notably reduce tracking complexity [9], [10], [11], such approaches do not naturally extend to other domains of high-dimensional data. Moreover, all of the aforementioned techniques are model-based, relying on full knowledge of the SS model, which is likely to be unavailable when tracking based on visual data.

In recent years, the combination of large-scale data-sets and advancements in deep learning has led to the development of several data-driven filtering methods, see review in [12]. These methods have shown empirical success in processing visual data, and typically involve deep neural network (DNN)

architectures, such as recurrent neural networks [13], attention mechanisms [14], and deep Markov models [15] for state tracking tasks. While these methods are based on architectures designed for generic time sequence processing, several DNN architectures were proposed specifically for tracking in SS models being inspired by model-based tracking algorithms [16], [17], [18], [19], [20], [21], [22], [23], resulting in, e.g., DNNs whose internal interconnection follows the flow of the EKF. Among these existing works, the systems of [20], [21], [22], [23] were specifically designed to cope with high-dimensional measurements, with a leading basis architecture being the recurrent Kalman network (RKN) of [20]. However, those methods suffer from difficulty in training, sensitivity to initialization, and generalization problems. In addition, while these data-driven works were inspired by model-based tracking algorithms, they do not leverage domain knowledge regarding the state evolution, even when such is available, as is the case in various applications including, e.g., localization and navigation [24, Ch. 6].

While the above data-driven approaches do not use knowledge about the SS model, one can combine model-agnostic deep-learning tools with SS-aware processing. A candidate approach to do so in the context of high-dimensional data is to use a DNN decoder to capture the complex measurement model [25], thus overcoming the need to analytically describe it, yet preserving the complexity associated with tracking using high-dimensional data. Alternatively, a widely adopted approach encodes the observations into a latent space via a DNN, i.e., using instead the inverse of the measurement model. These latent features are then used to track the state with, e.g., a conventional KF. This approach assumes that the latent features obey a simple SS model, typically a known Gaussian one in the latent domain as in [25], [26], [27], [28], [29], [30]. However, the resulting latent SS model is often non-Gaussian, which can impact the tracking accuracy in the latent space.

In this work, we propose Latent-KalmanNet, which addresses the difficulties of tracking high-dimensional data by simultaneously learning to track along with the latent space representation. To achieve this, we utilize the recently proposed KalmanNet [31], [32], [33], which learns from data to perform Kalman filtering in partially known SS models as a form of model-based deep learning [34], [35]. KalmanNet relies on a (possibly approximated) description of the measurement function. However, this information is unavailable in the setting considered in this work of complicated high-dimensional data, and the solution complexity grows with the dimensions of the measurements. Therefore, in Latent-KalmanNet we combine KalmanNet with latent-space encoding, and propose a novel training method that *jointly learns the latent representation and the filter operation*. Latent-KalmanNet uses a latent transformation assisted by its subsequent data-aided tracking method. The resulting latent representation is suitable for tracking while maintaining the interpretability and low complexity of the KF.

In particular, we first identify the main challenges associated with tracking from high-dimensional measurements in (1) the need to model stochasticity; (2) the operation with a possibly intractable measurement function; (3) the need to be applicable in real-time; and (4) the presence of possible mismatches in the state evolution model. Based on these challenges, we derive Latent-KalmanNet by gradually addressing each specific challenge, while accounting for settings where the state can be either partially observable or fully observable. The resulting Latent-KalmanNet combines two trainable components – an encoder that maps the observations into a latent representation, and KalmanNet, which tracks based on the latent features. Instead of designing these components separately, we exploit the ability of each module to facilitate the operation of its counterpart. Specifically, the tracking module is used to provide a prior for encoding, while the encoder generates a latent representation that is most suitable for tracking, and this desired behavior is learned from data using a dedicated alternating training mechanism. Our experimental study evaluates Latent-KalmanNet for tracking in challenging settings with high-dimensional visual data, identifying the benefits of each of the components incorporated in Latent-KalmanNet, while showing that the synergistic design of latent encoding and tracking yields notable performance improvements. Furthermore, we demonstrate that Latent-KalmanNet outperforms classic model-based nonlinear tracking algorithms as well as state-of-the-art deep architectures in terms of state estimation accuracy as well as inference speed.

The rest of the paper is organized as follows: Section II details the problem formulation and briefly describes the EKF and KalmanNet. Section III presents Latent-KalmanNet in a step-by-step manner, along with the proposed training method. Our numerical evaluation is provided in Section IV, while Section V concludes the paper.

Throughout the paper, we use boldface lower-case letters for vectors and boldface upper-case letters for matrices. The transpose, ℓ_2 norm, and gradient operator are denoted by $\{\cdot\}^\top$, $\|\cdot\|$, and $\nabla_{(\cdot)}$, respectively. Finally, $\mathbb{R}$ and $\mathbb{Z}^+$ are the sets of real and non-negative integer numbers, respectively.

II. System Model and Preliminaries

In this section, we present the system model and relevant preliminaries needed to derive Latent-KalmanNet in Section III. We start by formulating the problem of tracking in partially known SS models with high-dimensional observations in Subsection II-A. Then, in Subsections II-B–II-C, we briefly recall the model-based EKF and KalmanNet of [31], and identify their shortcomings for the considered setting.

A. Problem Formulation

We consider a dynamic system characterized by a (possibly) nonlinear, continuous SS model in discrete-time $t \in \mathbb{Z}^+$. Let $\mathbf{x}_t$ be the $m \times 1$ state vector, which evolves by a nonlinear state evolution function $\mathbf{f}(\cdot)$, and is driven by an additive zero-mean noise $\mathbf{e}_t$. The $n \times 1$ observation vector $\mathbf{y}_t$, is high-dimensional, and in particular $n \gg m$, that can be, e.g., the vector representation of an image/tensor[1]. The observed $\mathbf{y}_t$ is related to the state $\mathbf{x}_t$ via a complex and possibly unknown measurement function $\mathbf{h}(\cdot)$ (also known as observation or sensing function)

[1]For mathematical simplicity, we formulate our high-dimensional observations in vector form, which also represents tensor data by stacking their elements in vector form. The size n considered is the total number of elements in the observation.

with additive zero-mean noise $\mathbf{v}_t$. The resulting SS model is given by:

$$\mathbf{x}_t = \mathbf{f}(\mathbf{x}_{t-1}) + \mathbf{e}_t, \qquad \mathbf{x}_t \in \mathbb{R}^m, \tag{1a}$$

$$\mathbf{y}_t = \mathbf{h}(\mathbf{x}_t) + \mathbf{v}_t, \qquad \mathbf{y}_t \in \mathbb{R}^n. \tag{1b}$$

We consider a case where at least some of the state variables can be estimated from $\mathbf{y}_t$, which is related to the notion of observability, typically used in the context of deterministic systems [24, Ch. 3]. In particular, we use the term *fully observable* to denote measurement models where $\mathbf{y}_t$ is affected by all variables in $\mathbf{x}_t$, and all variables in state $\mathbf{x}_t$ can be recovered from $\mathbf{y}_t$ (i.e., the mapping $\mathbf{h}(\cdot)$ is injective). *partially observable* for models in which some of the entries of $\mathbf{x}_t$ cannot be recovered from $\mathbf{y}_t$ (though they may be dependent on $\{\mathbf{y}_\tau\}_{\tau\le t}$). That is, we examine both the fully observable case and the partially observable setting, where in the latter a single $\mathbf{y}_t$, can be used to recover only a subset of $p \le m$ variables in $\mathbf{x}_t$, denoted as the $p \times 1$ vector $\mathbf{P}\mathbf{x}_t$, with $\mathbf{P}$ being a $p \times m$ selection matrix. We henceforth focus our work on the partially observable setting as it also includes the fully observable setting by writing $\mathbf{P} = \mathbf{I}$ and $p = m$. We assume knowledge of which state variables can be estimated from $\mathbf{y}_t$ ($\mathbf{P}$ is given). This form of domain knowledge corresponds to a basic understanding of the underlying task, and the physical meaning of the state variables. For example, when tracking a moving object from visual data, one can typically extract position estimates from a given image, but not velocity or acceleration. However, it should be noted that this does not imply that the measurement function $\mathbf{h}(\cdot)$ is available.

Our goal is to develop a filtering algorithm for real-time state estimation, i.e., for the recovery of $\mathbf{x}_t$ from $\{\mathbf{y}_\tau\}_{\tau\le t}$ for each time instance t [36]. This algorithm should work effectively in both fully or partially observable SS models, while we assume that one has knowledge on which state variables are observable, i.e., $\mathbf{P}$ is known. The performance of a given estimator (obtained by the filtering approach) $\hat{\mathbf{x}}_t$ is measured using the mean-squared error (MSE), which is defined as $\mathbb{E}\{\|\hat{\mathbf{x}}_t - \mathbf{x}_t\|^2\}$.

While various methods have been proposed for tracking in SS models, our setting is associated with several challenges:

- C.1) The distribution of the noises, $\mathbf{e}_t$ and $\mathbf{v}_t$ in (1), is unknown and may be non-Gaussian as, e.g., stochasticity in visual data is often non-Gaussian.
- C.2) The available state-evolution function, $\mathbf{f}(\cdot)$, may be mismatched, e.g., obtained via a first-order linear approximation of complex physical dynamics, as is often the case in navigation and localization tasks [24, Ch. 6].
- C.3) The observations are high-dimensional ($n \gg m$), leading to high complexity and affecting real-time applicability.
- C.4) The measurement function $\mathbf{h}(\cdot)$ is unknown and possibly non-elementary.

To cope with the various unknown characteristics of (1), we are given access to a labeled data-set comprised of D trajectories of length T of paired observations and states,

$$\mathcal{D} \triangleq \left\{ \left\{ \mathbf{x}_t^{(d)}, \mathbf{y}_t^{(d)} \right\}_{t=1}^{T} \right\}_{d=1}^{D}. \tag{2}$$

Our proposed algorithm for tackling C.1-C.4 is detailed in Section III. Our design follows model-based deep-learning methodology [35], [34], where deep-learning tools are used to augment and empower model-based algorithms rather than replace them. Our method builds upon the KalmanNet architecture of [31], which augments the classic EKF, as briefly recalled in the next subsections.

B. Preliminaries: EKF

Various model-based filters have been developed for tracking in SS models (see, e.g. [3, Ch. 10]). One of the most common algorithms, which is suitable when the noises are Gaussian and the SS model is fully known, i.e., in the absence of Challenges C.1-C.4, is the EKF [37]. The EKF follows the operation of the KF, combining prediction based on the previous estimate with an update based on the current observation, while extending it to nonlinear SS models.

In particular, the EKF first predicts the next state and observation based on $\hat{\mathbf{x}}_{t-1}$ via

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{f}(\hat{\mathbf{x}}_{t-1}); \quad \hat{\mathbf{y}}_{t|t-1} = \mathbf{h}(\hat{\mathbf{x}}_{t|t-1}). \tag{3}$$

Then, the initial prediction is updated with a matrix $\mathcal{K}_t$, known as the Kalman gain (KG), which dictates the balance between relying on the state evolution function $f(\cdot)$ through (3) and the current observation $\mathbf{y}_t$. The estimate is computed as

$$\hat{\mathbf{x}}_t = \mathcal{K}_t \cdot \Delta\mathbf{y}_t + \hat{\mathbf{x}}_{t|t-1}, \tag{4}$$

where $\Delta\mathbf{y}_t \triangleq \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}$. The KG$\mathcal{K}_t$ is calculated via

$$\mathcal{K}_t = \hat{\mathbf{\Sigma}}_{t|t-1} \cdot \hat{\mathbf{H}}_t^\top \cdot \hat{\mathbf{S}}_{t|t-1}^{-1}, \tag{5}$$

where $\hat{\mathbf{\Sigma}}_{t|t-1}$ and $\hat{\mathbf{S}}_{t|t-1}$ are the covariance matrices of the state prediction $\hat{\mathbf{x}}_{t|t-1}$ and observation prediction $\hat{\mathbf{y}}_{t|t-1}$, respectively. These matrices are calculated via

$$\hat{\mathbf{\Sigma}}_{t|t-1} = \hat{\mathbf{F}}_t \cdot \hat{\mathbf{\Sigma}}_{t-1} \cdot \hat{\mathbf{F}}_t^\top + \mathbf{Q}, \tag{6}$$

$$\hat{\mathbf{S}}_{t|t-1} = \hat{\mathbf{H}}_t \cdot \tilde{\mathbf{\Sigma}}_{t|t-1} \cdot \hat{\mathbf{H}}_t^\top + \mathbf{R}, \tag{7}$$

where $\mathbf{Q}$ and $\mathbf{R}$ are the known covariance matrices of $\mathbf{e}_t$ and $\mathbf{v}_t$, respectively, for any $t \in \mathbb{R}$. The matrices $\hat{\mathbf{F}}_t$ and $\hat{\mathbf{H}}_t$ are instantaneous linearizations of $\mathbf{f}(\cdot)$ and $\mathbf{h}(\cdot)$, respectively, obtained i.e., using their Jacobian matrices evaluated at $\hat{\mathbf{x}}_{t-1}$ and $\hat{\mathbf{x}}_{t|t-1}$ (see [3, Ch. 10]), i.e.,

$$\hat{\mathbf{F}}_t = \nabla_{\mathbf{x}}\mathbf{f}(\hat{\mathbf{x}}_{t-1}); \quad \hat{\mathbf{H}}_t = \nabla_{\mathbf{x}}\mathbf{h}(\hat{\mathbf{x}}_{t|t-1}). \tag{8}$$

C. Preliminaries: KalmanNet

Challenges C.1-C.4 notably limit the applicability of the EKF for the setup detailed in Subsection II-A. KalmanNet proposed in [31] is designed to leverage data as in (2) to tackle Challenges C.1 and C.2 (but not C.3-C.4). In particular, KalmanNet builds on the insight that the missing and mismatched domain knowledge of the noise statistics and the linear approximations are encapsulated in the computation of the KG, $\mathcal{K}_t$ in (5). Consequently, it augments the EKF witha deep learning component

by replacing the computation of the KG with a dedicated RNN-based architecture. The features based on which the KG is computed are selected as ones that are indicative of the state and observation noise statistics. For example, these features can be chosen as $\Delta\mathbf{y}_t \triangleq \hat{\mathbf{y}}_t - \hat{\mathbf{y}}_{t|t-1}$ and $\Delta\mathbf{x}_t \triangleq \hat{\mathbf{x}}_t - \hat{\mathbf{x}}_{t-1}$. Accordingly, by letting $\mathbf{g}_{\boldsymbol{\theta}}^{\mathrm{KG}}(\cdot)$ denote the RNN mapping with parameters $\boldsymbol{\theta}$, KalmanNet sets the KG estimation to be:

$$\boldsymbol{\mathcal{K}}_t(\boldsymbol{\theta}) = \mathbf{g}_{\boldsymbol{\theta}}^{\mathrm{KG}}(\Delta\mathbf{y}_t, \Delta\mathbf{x}_{t-1}). \tag{9}$$

The rest of the filtering operation is the same via (3), while replacing the state estimate in (4) with

$$\hat{\mathbf{x}}_t = \boldsymbol{\mathcal{K}}_t(\boldsymbol{\theta}) \cdot \Delta\mathbf{y}_t + \hat{\mathbf{x}}_{t|t-1}. \tag{10}$$

The weights of the RNN, $\boldsymbol{\theta}$, were learned from a data-set (2), based on the regularized MSE loss function.

$$\mathcal{L}(\boldsymbol{\theta}) = \|\hat{\mathbf{x}}_t - \mathbf{x}_t\| + \lambda \|\boldsymbol{\theta}\|^2, \tag{11}$$

During training time, the gradients of (11) with respect to $\boldsymbol{\theta}$ are propagated through the entire EKF pipeline to learn the computation of the KG as the one yielding the state estimate best matching the data in the sense of the MSE loss from (11). By doing so, KalmanNet converts the EKF into a trainable discriminative model [38], where the data $\mathcal{D}$ is used to directly learn the KG, bypassing the need to enforce any model over the noise statistics and able to handle domain knowledge mismatches as in Challenges C.1-C.2. Moreover, KalmanNet preserves the interoperability of the KF, while being operable in partially known SS models; therefore, it allows to deduce uncertainty as shown in [32], and is amenable to training in an unsupervised manner [33].

Despite the ability of the KalmanNet architecture of [31] to learn from data to cope with Challenges C.1 and C.2, it is not suitable to be applied in our setting under Challenges C.3-C.4. In particular, the high dimension of the observations notably increases the complexity of its KG RNN and the resulting filter. Moreover, KalmanNet requires knowledge of $\mathbf{h}(\cdot)$, which is not analytically available in the current setting. This motivates the derivation of the proposed Latent-KalmanNet in the sequel.

III. LATENT-KALMANNET

In this section, we present the proposed Latent-KalmanNet algorithm, which tackles Challenges C.1-C.4. Our derivation of Latent-KalmanNet is presented in a step-by-step manner, where each step tackles an additional challenging aspect, while builds upon its preceding stages.

As noted in Subsection II-C, the main added Challenges considered here as compared to the setting for which KalmanNet is formulated in [31] are associated with the measurement model in (1b), i.e., Challenges C.3 and C.4. Therefore, our first step, detailed in Subsection III-A, considers an instantaneous estimation setting based solely on the measurement function. The second step, described in Subsection III-B, incorporates the state evolution function (1a) by adding a prior as an additional input, which accounts for temporal correlation and partial observability. Then, we unite the instantaneous estimate with the model-based EKF for tracking in Step 3 (Subsection III-C) to face Challenges C.3-C.4. Our fourth step, detailed in Subsection III-D, incorporates Challenges C.1 and C.2 by converting the joint instantaneous estimate and tracking algorithm into Latent-KalmanNet, by replacing EKF with KalmanNet. Latent-KalmanNet is learned as a trainable discriminative model, where data-driven tracking is done based on jointly learned latent features. We conclude our derivation with a discussion in Subsection III-E.

A. Step 1 - Instantaneous Estimate

We begin by considering the measurement function (1b) solely. The resulting task boils down to the instantaneous estimation of (the observable entries of) $\mathbf{x}_t$ from the measurement $\mathbf{y}_t$. The fact that the measurement model is high-dimensional (Challenge C.3) and unknown (Challenge C.4), combined with the availability of labeled data (2), motivates the usage of DNNs. A natural approach here is to use a regression DNN with parameters $\boldsymbol{\psi}$, denoted $\mathbf{g}_{\boldsymbol{\psi}}^{\mathrm{e}} : \mathbb{R}^n \mapsto \mathbb{R}^p$, and train it to map $\mathbf{y}_t$ into an estimate of the observable state variables $\mathbf{P}\mathbf{x}_t$. Where $\mathbf{P}$ is the selection matrix defined in II-A.

When properly trained with sufficient data, regression DNNs are often capable of learning to provide reliable estimates from high-dimensional data with complex statistical models [39, Ch. 13]. In particular, the DNN parameters $\boldsymbol{\psi}$ can be learned in a supervised manner via gradient-based optimization, e.g., stochastic gradient descent (SGD) and its variants. We adopt the regularized ℓ_2 norm MSE loss, which, for a given data-set $\mathcal{D}$, is computed as:

$$\mathcal{L}_{\mathcal{D}}^{\mathrm{e}}(\boldsymbol{\psi}) = \frac{1}{|\mathcal{D}|T} \sum_{d=1}^{|\mathcal{D}|} \sum_{t=1}^{T} \left\| \mathbf{g}_{\boldsymbol{\psi}}^{\mathrm{e}}\left(\mathbf{y}_t^{(d)}\right) - \mathbf{z}_t^{(d)} \right\|^2 + \lambda \|\boldsymbol{\psi}\|^2, \tag{12}$$

where $\lambda > 0$ is a regularization coefficient. The resulting instantaneous estimation system represents a straightforward data-driven approach to tackle the Challenges associated with the measurement model (1b). However, it does not account for the temporal correlation induced by the state evolution model (1a). Moreover, the full reliance on black box deep-learning architectures implies that the resulting system is sensitive to generalization problems and the model can easily overfit to the training set.

B. Step 2 - Incorporating the Evolution Model

The instantaneous estimator is oblivious to the partially known state evolution model in (1a). Therefore, by integrating the model in (1a), i.e., the (possibly approximated) state evolution function $\mathbf{f}(\cdot)$, one can potentially improve the estimation of the observable state variables provided by $\mathbf{g}_{\boldsymbol{\psi}}^{\mathrm{e}}(\cdot)$. Our rationale stems from viewing the inference task carried out by the DNN, i.e., recovering $\mathbf{P}\mathbf{x}_t$ from $\mathbf{y}_t$, as solving a non-convex optimization problem [35]. While tackling non-convex optimization is in general highly challenging, it can be greatly facilitated by providing a good initial guess, which hopefully lies in the proximity of the global optimum [40].

To incorporate the state evolution as a form of a prior, we apply $\mathbf{f}(\cdot)$ to the previous estimate, $\hat{\mathbf{x}}_{t-1}$. The previous estimate

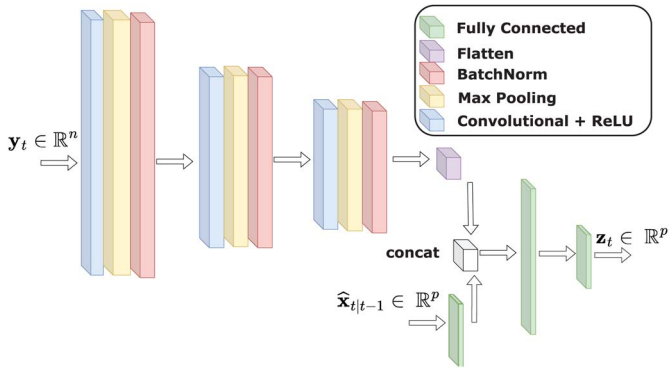


Fig. 1. Step 2 - Illustration of an encoder with prior implemented using a CNN, following the implementation utilized in the numerical study in Section IV.

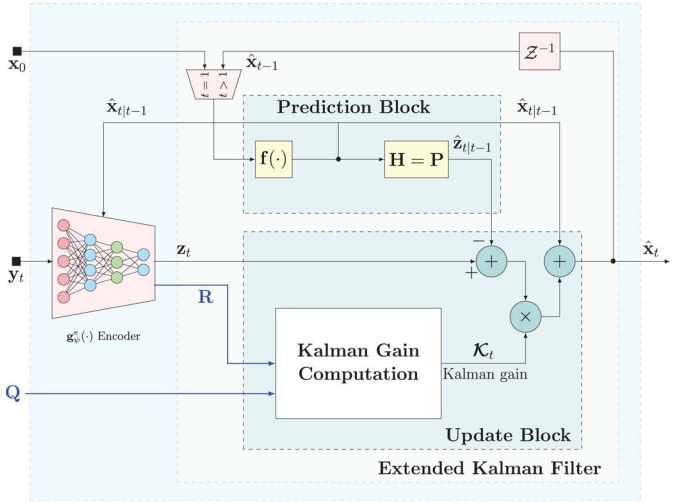


Fig. 2. Step 3 - block diagram of encoder with prior and EKF in cascade to track in latent space.

$\hat{\mathbf{x}}_{t-1}$ can be obtained from the previous encoder output, i.e., $\mathbf{g}^{\mathrm{e}}_{\psi}(\mathbf{y}_{t-1})$, while fused with an estimate of the unobservable variables, that can be provided as an initial guess and improve the instantaneous estimate by using the temporal correlation. An example of the resulting high-level architecture, where the prior prediction $\hat{\mathbf{x}}_{t|t-1}$ is provided to the DNN-based instantaneous estimate (implemented as a CNN) as additional features, is illustrated in Fig. 1. The DNN-based estimator now takes two multivariate inputs, $\hat{\mathbf{x}}_{t|t-1}$ and $\mathbf{y}_t$, and is thus mapping from $\mathbb{R}^n \times \mathbb{R}^m$ into $\mathbb{R}^p$:

$$\mathbf{z}_t = \mathbf{g}^{\mathrm{e}}_{\psi}(\mathbf{y}_t, \hat{\mathbf{x}}_{t|t-1}). \tag{13}$$

The encoder is trained using the regularized MSE loss as in (12). The incorporation of past outputs as a prior enables leveraging domain knowledge in the sense of the state evolution model, thus guiding the learning procedure towards a more desirable solution. At the same time, it also impacts the stability of the training procedure, as we have empirically observed in Subsection IV. Nonetheless, the learning procedure can be facilitated by exploiting the interpretability of the architecture, building upon the ability to view the prior $\hat{\mathbf{x}}_{t|t-1}$ as a noisy version of the desired state. One can thus set the prior during training to be the ground truth state with added noise, while choosing the noise magnitude based on an ablation study, as done in our numerical study reported in Section IV. Here, care should be taken not to push the encoder to learn solely from the observation when the noise is too large, while not relying only upon the prior where the noise is too small.

C. Step 3 - Joint Instantaneous Estimation and Tracking

Next, we assume that the relationship between the estimator output, $\mathbf{z}_t$, and the observable state variables, $\mathbf{P}\mathbf{x}_t$, can be represented as obeying a Gaussian distribution. This approach allows tracking based on the encoder's output, without accounting Challenges C.1 and C.2. In such cases, one can account for the temporal correlation by applying an EKF in cascade with the pretrained DNN encoder of Step 2. The rationale here is to assume that the DNN is properly trained such that its estimate approaches the minimal MSE estimator of $\mathbf{P}\mathbf{x}_t$ from $\mathbf{y}_t$. In such cases, the DNN output can be approximated as obeying

$$\mathbf{z}_t = \mathbf{g}^{\mathrm{e}}_{\psi}(\mathbf{y}_t, \hat{\mathbf{x}}_{t|t-1}) \approx \mathbf{P}\mathbf{x}_t + \tilde{\mathbf{v}}_t, \tag{14}$$

where $\tilde{\mathbf{v}}_t$ is zero-mean and mutually independent of $\mathbf{x}_t$. If $\tilde{\mathbf{v}}_t$ is also Gaussian and temporally independent, then (1a) along with (14) represent a (possibly nonlinear) Gaussian SS model, from which $\mathbf{x}_t$ can be tracked using the EKF. The second-order moment of $\tilde{\mathbf{v}}_t$, which is necessary for the KG computation (5), can be estimated from the validation error of the DNN encoder. The measurement matrix $\hat{\mathbf{H}}_t$ in this setting is set to $\hat{\mathbf{H}}_t = \mathbf{P}$. The system is illustrated in Fig. 2.

To apply the EKF while treating (14) as the measurement model, one should have knowledge of the distribution of the state noise $\mathbf{e}_t$, i.e., the matrix $\mathbf{Q}$. This can be estimated from the data $\mathcal{D}$. For instance, one can tune the dynamic noise variance to optimize performance by, e.g., assuming that $\mathbf{Q}$ is a scaled identity matrix and employing grid search to identify the variance parameter that yields the best performance on the available data. Alternatively, one can incorporate parametric estimation mechanisms, e.g., expectation maximization (EM) iterations, into the EKF [41]. The proposed cascaded operation allows utilizing a DNN to cope with the challenging measurement model while systematically incorporating the state evolution model. This is achieved by separating instantaneous estimation from the tracking task, where the temporal correlation is exploited. Jointly treating the instantaneous estimate task along with its subsequent tracking allows for improving the overall performance, as shown in Section IV. Nonetheless, the fact that an EKF is utilized over the approximated SS model, together with possibly partially known dynamics, implies that there might be a mismatch in the setting - Challenges C.1 and C.2. It can be an inaccurate dynamic function $\mathbf{f}(\cdot)$, gap in the sampling rate, or that the DNN estimation error $\tilde{\mathbf{v}}_t$ is inaccurate for using it as an input for the EKF pipeline. This motivates our final step, replacing the EKF with KalmanNet, which formulates Latent-KalmanNet.

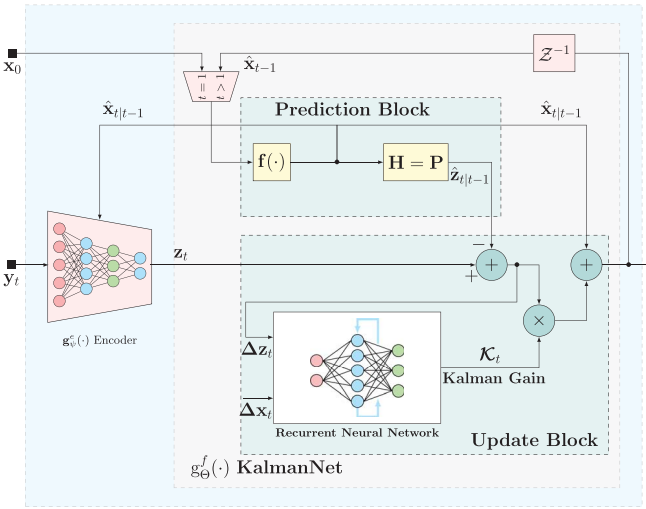


Fig. 3. Step 4 - latent-KalmanNet block diagram.

D. Step 4 - Latent-KalmanNet

The system detailed in Step 3 builds upon the insight that the relationship between the encoder's output $\mathbf{z}_t$ and the state $\mathbf{x}_t$ obeys an (approximated) SS model given by (1a) and (14). We conclude our design by accounting for Challenges C.1 and C.2, and the fact that the error term in (14) is likely to obey an unknown distribution. This motivates using KalmanNet instead of the EKF, which is particularly suitable for filtering in such settings, and bypasses the need to impose a specific distribution on the noise terms in the SS model. The resulting algorithm, encompassing the architecture and its training procedure detailed next, is coined *Latent-KalmanNet*.

Architecture: To formulate the system operation, we let $\boldsymbol{\theta}$ be the internal RNN parameters of KalmanNet, which implements a mapping $\mathbf{g}^{\mathrm{f}}_{\boldsymbol{\theta}}:\mathbb{R}^p\mapsto\mathbb{R}^m$ with state-evolution function $\mathbf{f}(\cdot)$ and measurement function given by $\mathbf{h}(\mathbf{x})=\mathbf{P}\mathbf{x}$. As detailed in Subsection II-C, KalmanNet uses the previous estimate $\hat{\mathbf{x}}_{t-1}$ to predict the next state as $\hat{\mathbf{x}}_{t|t-1}=\mathbf{f}\left(\hat{\mathbf{x}}_{t-1}\right)$. This prediction is then used as the prior provided to the encoder of Step 2, producing $\mathbf{z}_t$ via (13). The estimate of $\mathbf{x}_t$ is written as

$$\hat{\mathbf{x}}_t=\mathbf{g}^{\mathrm{f}}_{\boldsymbol{\theta}}(\mathbf{z}_t,\hat{\mathbf{x}}_{t-1}). \tag{15}$$

The resulting architecture is illustrated in Fig. 3, where the two modules, the encoder and KalmanNet, can be viewed as an encoder-decoder pair. The encoder's output $\mathbf{z}_t$ thus serves as the *latent representation*, following the conventional autoencoder terminology [42]. Specifically, the encoder and decoder (KalmanNet) aid one another by providing a low-dimensional latent representation (by the encoder), and a prior for obtaining the latent (by KalmanNet). The training procedure, detailed below, encourages the overall encoder-decoder to have the latent representation $\mathbf{z}_t$ be one that leads to the most accurate state estimate at the output of KalmanNet, and not necessarily the most reliable estimate of $\mathbf{P}\mathbf{x}_t$ (as in Steps 1-3). Once trained, the estimation procedure on each time step, during inference, is summarized as Algorithm 1.

Algorithm 1: Latent-KalmanNet Inference

Init: Trained encoder $\boldsymbol{\psi}$; Trained KalmanNet $\boldsymbol{\theta}$
Input: Observations $\mathbf{y}_t$; previous estimate $\hat{\mathbf{x}}_{t-1}$
Predict $\hat{\mathbf{x}}_{t|t-1}=\mathbf{f}(\hat{\mathbf{x}}_{t-1})$;
Predict $\hat{\mathbf{z}}_{t|t-1}=\mathbf{P}\hat{\mathbf{x}}_{t|t-1}$;
Encode observations via $\mathbf{z}_t=\mathbf{g}^{\mathrm{e}}_{\boldsymbol{\psi}}(\mathbf{y}_t,\hat{\mathbf{x}}_{t|t-1})$;
Apply the RNN $\boldsymbol{\theta}$ to compute KG $\mathcal{K}_t$;
Estimate via $\hat{\mathbf{x}}_t=\mathbf{g}^{\mathrm{f}}_{\boldsymbol{\theta}}(\mathbf{z}_t)=\mathcal{K}_t\cdot(\mathbf{z}_t-\hat{\mathbf{z}}_{t|t-1})+\hat{\mathbf{x}}_{t|t-1}$;
return $\hat{\mathbf{x}}_t$

Training: The proposed architecture is a concatenation of two modules: A DNN estimator $\mathbf{g}^{\mathrm{e}}_{\boldsymbol{\psi}}(\cdot)$ and KalmanNet $\mathbf{g}^{\mathrm{f}}_{\boldsymbol{\theta}}(\cdot)$. Both are differentiable [31], allowing the overall architecture, parameterized by $(\boldsymbol{\theta},\boldsymbol{\psi})$, to be trained end-to-end as a discriminative model [38]. We use the ℓ_2 regularized MSE loss, which for a given data-set $\mathcal{D}$ is evaluated as

$$\mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta},\boldsymbol{\psi})=\frac{1}{|\mathcal{D}|T}\sum_{d=1}^{|\mathcal{D}|}\sum_{t=1}^{T}\mathcal{L}_t^{(d)}(\boldsymbol{\theta},\boldsymbol{\psi})+\lambda_1\left\|\boldsymbol{\theta}\right\|^2+\lambda_2\left\|\boldsymbol{\psi}\right\|^2, \tag{16}$$

where $\lambda_1,\lambda_2>0$ are regularization coefficients. The loss term for each time step in a given trajectory of (16) is computed as

$$\mathcal{L}_t^{(d)}(\boldsymbol{\theta},\boldsymbol{\psi})=\left\|\hat{\mathbf{x}}_t^{(d)}-\mathbf{x}_t^{(d)}\right\|^2, \tag{17}$$

where the model output is computed via (10), (13), and (15):

$$\begin{aligned}\hat{\mathbf{x}}_t^{(d)}&=\mathbf{g}^{\mathrm{f}}_{\boldsymbol{\theta}}\left(\mathbf{g}^{\mathrm{e}}_{\boldsymbol{\psi}}\left(\mathbf{y}_t^{(d)},\mathbf{f}\left(\hat{\mathbf{x}}_{t-1}^{(d)}\right)\right),\hat{\mathbf{x}}_{t-1}^{(d)}\right)\\&=\hat{\mathbf{x}}_{t|t-1}^{(d)}+\mathcal{K}_t(\boldsymbol{\theta})\cdot(\mathbf{z}_t^{(d)}-\hat{\mathbf{z}}_{t|t-1}^{(d)}).\end{aligned} \tag{18}$$

The loss measure in (16) builds upon the ability to back-propagate the loss to the computation of the KG$\mathcal{K}_t$ [43]. In particular, one can obtain the loss gradient of a given trajectory d in given time step t with respect to the KG from the output $\hat{\mathbf{x}}_t^{(d)}$ of Latent-KalmanNet since (17) and (18) implies that:

$$\begin{aligned}\frac{\partial\mathcal{L}_t^{(d)}(\boldsymbol{\theta},\boldsymbol{\psi})}{\partial\mathcal{K}_t}&=\frac{\partial\left\|\mathcal{K}_t\Delta\mathbf{z}_t-\Delta\mathbf{x}_t\right\|^2}{\partial\mathcal{K}_t}\\&=2(\mathcal{K}_t\Delta\mathbf{z}_t-\Delta\mathbf{x}_t)\cdot\Delta\mathbf{z}_t^{\top},\end{aligned} \tag{19}$$

where $\Delta\mathbf{x}_t=\mathbf{x}_t^{(d)}-\hat{\mathbf{x}}_{t|t-1}^{(d)}$, and $\Delta\mathbf{z}_t=\mathbf{z}_t^{(d)}-\hat{\mathbf{z}}_{t|t-1}^{(d)}$. The gradient computation in (19) indicates that one can learn the computation of the KG by training Latent-KalmanNet end-to-end. This allows training the overall filtering system, including both the latent encoding and its tracking into the state without having to externally provide ground truth values of the KG or of the latent features for training purposes. The fact that the MSE loss in (17) is computed based on the output of KalmanNet rather than that of $\mathbf{g}^{\mathrm{e}}_{\boldsymbol{\psi}}(\cdot)$, implies that the latter will not necessarily learn to estimate the observable state variables $\mathbf{P}\mathbf{x}_t$, as in when training via (12). Instead, it is trained to encode the high-dimensional observations $\mathbf{y}_t$ (along with the prior $\hat{\mathbf{x}}_{t|t-1}$) into *latent features*, from which KalmanNet can most reliably recover the state. For this reason, we coin the algorithm *Latent-KalmanNet*.

Algorithm 2: Latent-KalmanNet Alternating Training

Init: Fix learning rates $\mu_1, \mu_2 > 0$ and epochs $i_{\max}$
Input: Training set $\mathcal{D}$
`Warm start:`
for $i = 0, 1, \ldots, i_{\max} - 1$ **do**
  Randomly divide $\mathcal{D}$ into Q batches $\{\mathcal{D}_q\}_{q=1}^{Q}$;
  for $q = 1, \ldots, Q$ **do**
    Compute batch loss $\mathcal{L}^{\mathrm{e}}_{\mathcal{D}_q}(\boldsymbol{\psi})$ by (12);
    Update $\boldsymbol{\psi} \leftarrow \boldsymbol{\psi} - \mu_1 \nabla_{\boldsymbol{\psi}} \mathcal{L}^{\mathrm{e}}_{\mathcal{D}_q}(\boldsymbol{\psi})$;
`Alternating minimization:`
for $i = 0, 1, \ldots, i_{\max} - 1$ **do**
  Randomly divide $\mathcal{D}$ into Q batches $\{\mathcal{D}_q\}_{q=1}^{Q}$;
  for $q = 1, \ldots, Q$ **do**
    Compute batch loss $\mathcal{L}_{\mathcal{D}_q}(\boldsymbol{\theta}, \boldsymbol{\psi})$ by (16);
    Update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \mu_2 \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{D}_q}(\boldsymbol{\theta}, \boldsymbol{\psi})$;
  for $q = 1, \ldots, Q$ **do**
    Compute batch loss $\mathcal{L}_{\mathcal{D}_q}(\boldsymbol{\theta}, \boldsymbol{\psi})$ by (16);
    Update $\boldsymbol{\psi} \leftarrow \boldsymbol{\psi} - \mu_1 \nabla_{\boldsymbol{\psi}} \mathcal{L}_{\mathcal{D}_q}(\boldsymbol{\theta}, \boldsymbol{\psi})$;
return $(\boldsymbol{\theta}, \boldsymbol{\psi})$

Latent-KalmanNet enables joint learning of $(\boldsymbol{\theta}, \boldsymbol{\psi})$ via gradient-based optimization, e.g., mini-batch SGD and its variants, such as Adam or RMSProp. However, carrying this out in practice can be challenging and often unstable, as the learning procedure needs to simultaneously tune the latent representation and the corresponding KG computation. In addition, one can suggest the estimation of second-order derivatives for alternative learning procedures [44], though possibly increased computational burden. Nonetheless, the fact that the architecture is decomposable into distinct trainable building blocks with concrete tasks facilitates training via alternating optimization. This is achieved by iteratively optimizing the filter $\boldsymbol{\theta}$ while freezing $\boldsymbol{\psi}$, followed by training of the latent representation $\boldsymbol{\psi}$ which best fits the filter with fixed weights $\boldsymbol{\theta}$ based on (16). Additionally, one can initially train the observable variables of the encoder module separately, via the regularized ℓ_2 norm loss (12). This form of modular training [45] constitutes a warm start which is empirically shown to facilitate learning. The resulting procedure is summarized as Algorithm 2.

E. Discussion

The proposed Latent-KalmanNet is designed to tackle the challenges of tracking from complex high-dimensional observations under partially known dynamics. It leverages data to enable reliable tracking, overcoming the missing knowledge of the measurement function and the noise statistics. Latent-KalmanNet is derived in gradual steps obtained from pinpointing the specific challenges associated with the filtering problem detailed in Subsection II-A. In particular, the usage of a DNN trained in a supervised manner for coping with the complex measurement model in Step 1 is a straightforward approach for instantaneous estimations. Its cascading with an EKF is a natural extension for incorporating temporal correlation, and a similar approach of applying an EKF to data-driven extracted features was also proposed in previous works, e.g., [26], [25].

TABLE I
CHALLENGES COMPARISON

Challenge	EKF	KalmanNet	Black-Box DNNs	Latent-KalmanNet
C.1	X	V	V	V
C.2	X	V	V	V
C.3	X	X	V	V
C.4	X	X	V	V
Use $\mathbf{f}(\cdot)$	V	V	X	V

However, the usage of the evolution model $\mathbf{f}(\cdot)$ in Step 2 for improving instantaneous estimate due to temporal correlation; replacing the EKF with the trainable KalmanNet in Step 4; and the formulation of a suitable training procedure which encourages both modules to facilitate their counterpart's operation in tracking, are novel aspects of our design. These components are particularly tailored to cope with the challenging partially known SS model in (1), without enforcing a model on the noise statistics and the measurement function, while being geared towards real-time applications with low-latency inference demands.

As summarized in Table I, Latent-KalmanNet is designed to tackle Challenges C.1-C.4 while extending upon the EKF (which requires knowledge of the SS model thus struggles in presence of Challenges C.1-C.4) and KalmanNet of [31] (whose operation does not cope with Challenges C.3-C.4). While black-box architectures, e.g., RNNs as in [20], can also cope with Challenges C.3-C.4, being invariant of the SS model, Latent-KalmanNet also preserves the interpretable flow of the EKF and its principled incorporation of the available domain knowledge.

Compared with the preliminary findings of this research reported in [1], the Latent-KalmanNet algorithm presented here is not restricted to using instantaneous estimators for latent feature extraction, and can in fact learn to contribute to the latent state encoding. Furthermore, while [1] was only applicable in fully observable SS models, Latent-KalmanNet presented here is designed to leverage its access to the state evolution model to track the state also in partially observable settings. This enables its application in various challenging scenarios, as also demonstrated in our numerical study reported in Section IV.

Our design of Latent-KalmanNet improves estimation performance by breaking the separation between feature extraction and filtering. In principle, one can claim that providing the prior $\hat{\mathbf{x}}_{t|t-1}$ effectively delegates the filtering task to the instantaneous estimate DNN for fully observable models, and renders the following filtering step meaningless. However, our numerical findings reported in Section IV demonstrate that this is not the case, and that the system benefits from both prior-aided instants estimation as well as from the filtering operation based on that estimate. Step 4 allows the resulting algorithm to operate without enforcing a Gaussian SS model on the latent representation, as opposed to [25]. This is obtained as Algorithm 1 bypasses the need to model the stochasticity in the SS model by using KalmanNet [31]. Unlike state-of-the-art DNN-aided filters, such as the RKN of [20], we do not replace all

the KF procedures with DNNs and preserve the operation of the model-based filter. This allows us to systematically incorporate the available domain knowledge on the state evolution, improving performance and generalization, as demonstrated in Section IV.

The hybrid model-based/data-driven design of Latent-KalmanNet yields gains not only in accuracy and interpretability. It can also achieve faster inference speed compared to other model-based solutions or highly parameterised data driven models, while supporting training with relatively limited datasets, as demonstrated in Section IV. The computation complexity for each time step is linear in the dimensions of the trainable modules, being the complexity order of inferring using a DNN, while its augmentation with the classic EKF enables using relatively compact architectures, as we do in Section IV. Moreover, while Latent-KalmanNet follows the operation of the EKF, it does not involve Jacobian computations as in (8) or matrix inversion as in (5) on each time step. This implies that Latent-KalmanNet is a good candidate to apply for high-dimensional SS models and on computationally limited devices, compared to other model-based solutions, as well as data-driven approaches with a large volume of weights.

Latent-KalmanNet leverages data to adaptively learn the KG, alongside the learned latent features. The choice to use RNN-based architectures stems from the suitability of such DNNs in processing time sequences, combined with their relative compactness in terms of the number of trainable parameters. Specifically, the usage of relatively compact architecture facilitates the implementation of Latent-KalmanNet for real-time inference on edge devices [46]. Alternative architectures, e.g., based on transformers [47], can also be considered, though they are expected to come at the cost of increased parameterization, complexity, and latency compared to our RNN-based architecture. While the KG-computing DNN architecture is completely invariant of the distribution of the noise signals, designed to comply with Challenge C.1, if one has prior statistical knowledge, it can be incorporated into the neural architecture. For instance, one can combine different knowledge of the noise covariance into Architecture 2 of [31], or use the DNN flow proposed in [48]. We leave the study of these extensions of Latent-KalmanNet for future study.

One can possibly pretrain the encoder in an unsupervised manner using an autoencoder architecture, as done in [27], aiming to learn a compact representation of input data within the latent space, which can efficiently capture the essential information required to reconstruct the input data. However, our selection of the latent representation in (14) is based on domain knowledge, as it is particularly set such that the latent $\mathbf{z}_t$ is viewed as a noisy representation of the state variables that can be recovered from a single observation. This design follows our model-based deep learning methodology, which leads to an interpretable and simple DNN-aided system. While one can consider treating the latent space dimension as a hyperparameter (possibly lifting its dimensionality beyond that of the state), and learn $\mathbf{P}$ from data, while the encoder performs a different role, as a compression algorithm or a dimensionality reduction technique. Such an extension would affect the interpretability of Latent-KalmanNet, and is thus left for future investigation. Moreover, preserving the model-based operation of the KF was shown to bring operational gains beyond accuracy and training complexity. For instance, it was shown in [32] to enable extracting uncertainty on the estimates, and in [33] to facilitate unsupervised learning. We leave the exploration of these latent-space learned filtering for future work.

IV. EMPIRICAL STUDY

In this section, a comprehensive numerical analysis is performed[2] on the proposed Latent-KalmanNet to assess its performance. We consider two setups involving the tracking of a dynamic system from visual measurements: The first study detailed in Subsection IV-A considers is a partially observable dynamic system representing the tracking of a pendulum. It is used to critically examine the design steps of Latent-KalmanNet and to evaluate the contribution of each of its components. The second study, presented in Subsection IV-B, considers the Lorenz attractor chaotic system, which is an observable dynamic system. This setup is used to compare our proposed Latent-KalmanNet against both model-based and data-driven techniques across various scenarios, with both full and partial domain knowledge. It should be noted that since KalmanNet is not designed to tackle Challenges C.3 and C.4, it is computationally infeasible for the considered scenarios.

A. Pendulum Data

We commence our numerical study by comprehensively evaluating the impact of each design step outlined in Section III over the Pendulum setting, further detail in the following.

1) SS Model: In the considered SS model, the vector $\mathbf{x}_t$ represents the state of an oscillating pendulum, encompassing both the angle ϕ_t and the angular velocity ω_t, i.e., $\mathbf{x}_t = [\phi_t, \omega_t]^\top$. We focus on tracking the angle ϕ_t along the trajectory of a pendulum movement that is released from rest at a pre-defined point, i.e., the MSE is reported with respect to ϕ_t. The state evolution model of the pendulum is defined by mechanical system laws, making it highly nonlinear in nature, as given in the following equation

$$\mathbf{x}_t = \begin{bmatrix} 1 & \Delta_t \\ 0 & 1 \end{bmatrix} \cdot \mathbf{x}_{t-1} - \frac{g}{\ell} \cdot \begin{bmatrix} 1/2 \cdot \Delta_t^2 \\ \Delta_t \end{bmatrix} \cdot \sin(\phi_{t-1}) + \mathbf{e}_t. \quad (20)$$

In (20), Δ_t denotes the sampling interval, dictating the time difference between consecutive observations, and $\mathbf{e}_t$ is an i.i.d. zero-mean Gaussian noise with covariance $\mathbf{Q} = q^2 \cdot \mathbf{I}$, where $q^2 = 0.1$. The gravitational acceleration is set to a constant value of $g = 9.81\ [\mathrm{m/sec^2}]$, and the length of the string is represented by ℓ. Fig. 4 illustrates the physical setup.

The observations $\mathbf{y}_t$ are 28×28 gray-scale images generated from the sampled trajectories of the pendulum. The images capture the pendulum's dynamic movements as if they were taken by a camera set in front of the system, corrupted by i.i.d. Gaussian observation noise $\mathbf{v}_t$ with covariance $\mathbf{R} = r^2 \cdot \mathbf{I}$,

[2]The source code and hyperparameters used are available at https://github.com/KalmanNet/Latent_KalmanNet_TSP.git

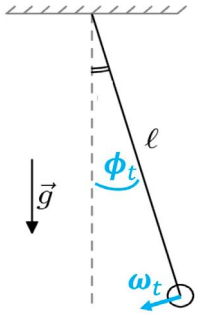


Fig. 4. Pendulum: physical setup and state variables.

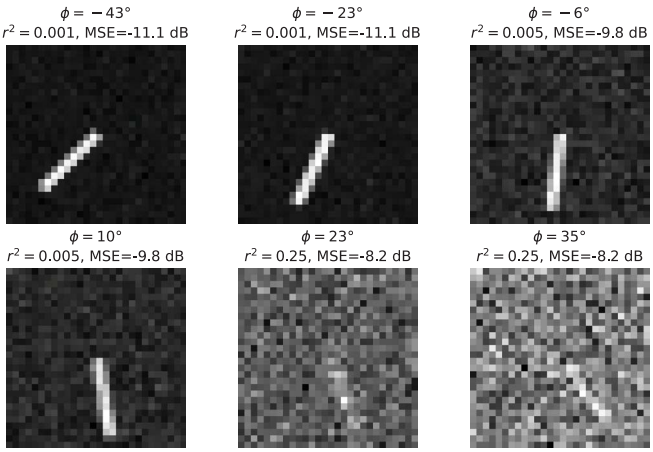


Fig. 5. Pendulum: several representative gray-scale observations along with their corresponding angle variable ϕ. In addition, the Gaussian noise magnitude r^2 that was added to the image, and the MSE achieved by Latent-KalmanNet.

where $r^2 \in 0.001, \ldots, 0.25$. Fig. 5 shows several representative visual observations of a given trajectory, with different added noise variance r^2. As only the angle can be recovered from a single image, this setting represents a partially observable SS model (see Subsection II-A) with $\mathbf{P} = [1, 0]$. We use this model to generate $D = 1,000$ trajectories of length $T = 200$ which comprise the data-set $\mathcal{D}$ as (2), with additional 100 trajectories for evaluation.

2) Tracking Methods: To this end, we evaluate the following approaches, representing the design steps in developing Latent-KalmanNet:

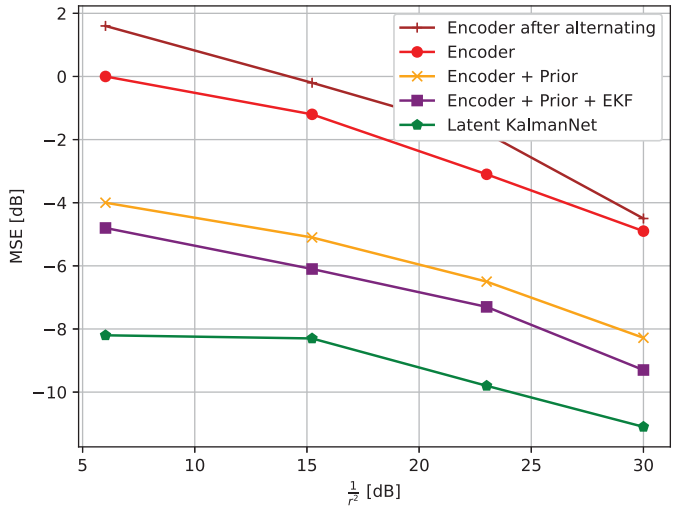
- *Encoder* (Step 1): We implement a purely data-driven, model-agnostic convolutional encoder, comprised of 3 convolutional layers, followed by 2 fully connected (FC) layers, and $p = 1$ output neuron, as detailed in Table II. The chosen architecture, and particularly its usage of ReLU activation functions, was inspired by previous works in literature for feature extracting related tasks - image denoising [49], [50], and object detection [51]. The encoder is trained in a supervised manner on the data-set $\mathcal{D}$ (2) with loss function (12) and Adam optimizer to map each $\mathbf{y}_t$ into the observable state variable ϕ_t.

TABLE II
ENCODER ARCHITECTURE

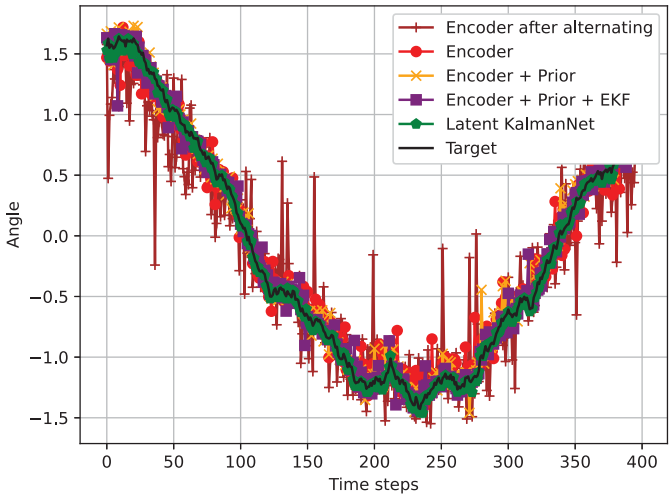
Layer	Filter Size	Stride	Channels	Output Size
Input	-	-	-	1×28×28
conv2D	3×3	2	8	8×14×14
ReLU	-	-	-	8×14×14
Batch Norm	-	-	8	8×14×14
conv2D	3×3	2	16	16×7×7
ReLU	-	-	-	16×7×7
Batch Norm	-	-	16	16×7×7
conv2D	3×3	2	32	32×4×4
ReLU	-	-	-	32×4×4
Batch Norm	-	-	32	32×4×4
Flatten	-	-	-	512
FC	-	-	32	32
ReLU	-	-	-	32
FC	-	-	p	p

- *Encoder + Prior* (Step 2): We modify the encoder architecture, detailed in Table II by incorporating a prior, $\hat{\mathbf{x}}_{t|t-1} = \mathbf{f}(\hat{\mathbf{x}}_{t-1})$, as an additional input to the observed image $\mathbf{y}_t$. The prior undergoes an FC layer and concatenates to the flattened version of the extracted features, as illustrated in Fig. 1. The output of this encoder still represents the estimate of ϕ_t, but is fused with an estimate of unobservable angular velocity variable, obtained by applying the state evolution function.
- *Encoder + Prior + EKF* (Step 3): On top of the trained encoder with prior of Step 2, we apply an EKF with the measurement function set to $\mathbf{h}(\mathbf{x}) = \mathbf{P}\mathbf{x}$. The variance of the state evolution noise is selected through grid search, while the observation noise is determined by the empirical estimate loss at the output of the encoder.
- *Latent-KalmanNet* (Step 4): The proposed Latent-KalmanNet uses the Encoder + Prior of Step 2, and combines it with KalmanNet based implemented using Architecture 2 of [31]. Latent-KalmanNet is trained using Algorithm 2 with Adam optimizer.

3) Results: In Fig. 6(a) we compare the MSE averaged over 100 test trajectories achieved by the considered methods in recovering angle, i.e., the observable entry of the state variables. The findings in Fig. 6(a) reveal the individual contribution of each of the design steps comprising Latent-KalmanNet. There, it is shown that including a prior based on the state evolution model notably improves the performance of an encoder trained solely over the observations. Moreover, using this estimate as latent features and employing EKF tracking in latent space further improves performance, though less dramatically; However, replacing the EKF with KalmanNet that is jointly trained (alternately) along with the latent representation as Latent-KalmanNet achieves substantial improvements in MSE. We also depict in Fig. 6(a) the ablation study of alternating training, where the results marked as 'Encoder' are for the pre-trained, standalone Encoder, and the results marked as 'Encoder after alternating' are for an encoder trained using Algorithm 2 and evaluated independently, rather than for the entire system, which would include KalmanNet and marked as 'Latent-KalmanNet'. The latent representation learned is less accurate in estimating



(a) MSE vs. different Gaussian noise variance added to the images.



(b) State estimation of the angle variable for a single trajectory

Fig. 6. Pendulum: design steps contribution.

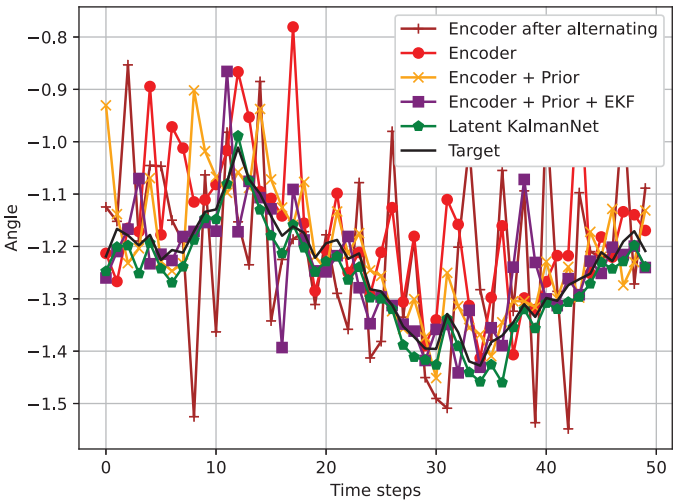


Fig. 7. Zoom in of the first 50 to time steps.

the state compared to an instantaneous encoder. However, this representation is learned such that it facilitates tracking in latent space, as evidenced by the superior performance of Latent-KalmanNet. In Fig. 6(b) the superiority of Latent-KalmanNet is showcased when tracking a single trajectory. There, the improved tracking quality is highlighted by observing a trajectory spanning 400 time instances. To complement this assessment, a zoomed-in version of the first 50 time steps estimation results is presented in Fig. 7. These results demonstrate that Latent-KalmanNet's principled incorporation of the state evolution model knowledge while jointly learning the latent representation with the tracking algorithm results in improved estimation and smoother tracking.

B. Comparative Evaluation of Latent-KalmanNet

Next, we present a comprehensive numerical evaluation of Latent-KalmanNet and its performance in terms of MSE and latency. To that aim, we simulate various scenarios involving the nonlinear Lorenz system model detailed in the following.

1) SS Model: Here, the state vector $\mathbf{x}_t$ is a three-dimensional chaotic solution to the Lorenz system of ordinary differential equations. The system describes chaotic particle movement sampled into discrete time intervals [52]. The result is a non-linear state evolution model of the continuous-time process, showcasing the dynamic interplay of forces shaping the chaotic particle's movement. The noise-free state-evolution equation is obtained from the differential equation

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{A}(\mathbf{x}_t)\cdot\mathbf{x}_t; \quad \mathbf{A}(\mathbf{x}_t) = \begin{bmatrix} -10 & 10 & 0 \\ 28 & -1 & -x_1 \\ 0 & x_1 & -8/3 \end{bmatrix} \tag{21}$$

The model is converted into a discrete-time state-evolution model by repeating the steps used in [53]. First, we sample the noiseless process with sampling interval Δ_t and assume that can be kept constant in a small neighborhood of $\mathbf{x}_t$; i.e.,

$$\mathbf{A}(\mathbf{x}_t) \approx \mathbf{A}(\mathbf{x}_{t+\Delta_t}). \tag{22}$$

Then, the continuous-time solution of the differential system (21), which is valid in the neighborhood of $\mathbf{x}_t$ for a short time interval Δ_t, is

$$\mathbf{x}_{t+\Delta_t} = \exp(\mathbf{A}(\mathbf{x_t})\cdot\Delta t)\cdot\mathbf{x}_t. \tag{23}$$

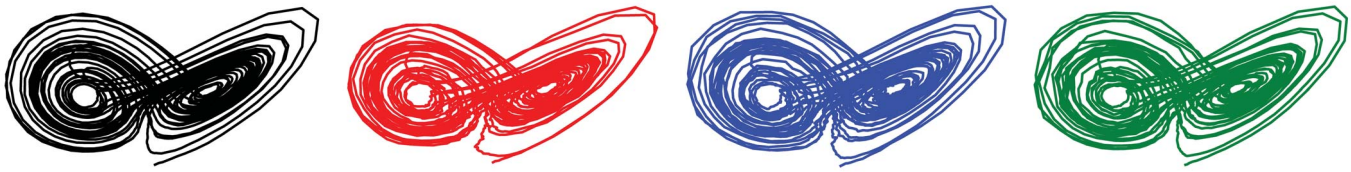
Finally, we take the Taylor series expansion of (23) and a finite series approximation (with J coefficients), which results

$$\mathbf{F}(\mathbf{x}_t) \triangleq \exp(\mathbf{A}(\mathbf{x}_t)\cdot\Delta t) = \mathbf{I} + \sum_{j=1}^{J} \frac{(\mathbf{A}(\mathbf{x}_t)\cdot\Delta_t)^j}{j!} \tag{24}$$

The resulting discrete-time evolution process is given by

$$\mathbf{x}_{t+1} = \mathbf{f}(\mathbf{x}_t) = \mathbf{F}(\mathbf{x}_t)\cdot\mathbf{x}_t. \tag{25}$$

The discrete-time state-evolution model presented in (25), is augmented with additional zero-mean Gaussian noise with i.i.d. entries of variance $q^2 = 0.005$, obtaining a noisy state-evolution representation as in (1a). An illustration of the state trajectory is given in the left side of Fig. 8.



(a) Ground truth. (b) Encoder (c) RKN (d) Latent-KalmanNet

Fig. 8. Lorenz system: ground truth trajectory of the state vs. trajectories estimated with the different methods.

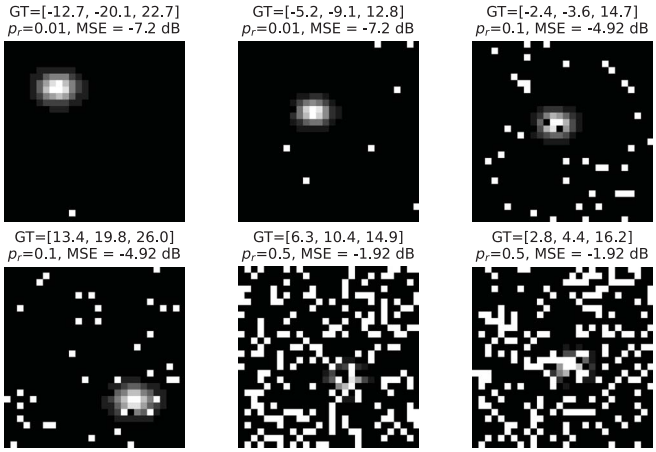


Fig. 9. Lorenz system: several representative gray-scale observations along with their corresponding state $\mathbf{x}_t$, set to be the ground truth (GT). In addition, the S&P noise probability, and the MSE achieve by Latent-KalmanNet.

We emulate visual representation of the movement described by $\mathbf{x}_t$ in the form of 28×28 matrices. To represent visual observations used in particle tracking, e.g., [54], the measurement function evaluated at coordinate $\mathbf{c} \in \mathbb{R}^2$ for state value $\mathbf{x} = [x_1, x_2, x_3]^\top$ is modeled a Gaussian point spread function whose intensity depends on the lateral state coordinate, where we use

$$\mathbf{h}(\mathbf{c}; \mathbf{x}) = 10 \exp\left(\frac{-1}{2x_3} \left\| \mathbf{c} - \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right\|^2 \right). \tag{26}$$

The observations are corrupted by Salt and Pepper (S&P) noise, modeled as an i.i.d. scaled Bernoulli vector with probability p_r. This type of noise is common in digital images and can be caused by sharp and sudden disturbances in the image signal when transmitting images over noisy digital channels. Representative visual observations of a given trajectory are depicted in Fig. 9. All considered tracking algorithms have access to the same data-set (2). Unless otherwise stated, the data was generated from the Lorenz system SS model with Taylor order of $J = 5$, sampling interval of $\Delta_t = 0.02$, and a trajectory length of $T = 200$.

2) Tracking Methods: The following experiments aim to assess the efficacy of Latent-KalmanNet in comparison to benchmark data-driven algorithms as well as with model-based tracking in latent space. In particular, we evaluate the following tracking algorithms:

1) *Encoder:* A data-driven convolutional encoder as in Table II, i.e., comprised of three convolutional layers and two FC layers with $p = 3$ output neurons.

TABLE III
LORENZ SYSTEM: NUMERIC MSE VALUES FOR THE SETTING REPORTED IN FIG. 10(a), INCLUDING STANDARD DEVIATION

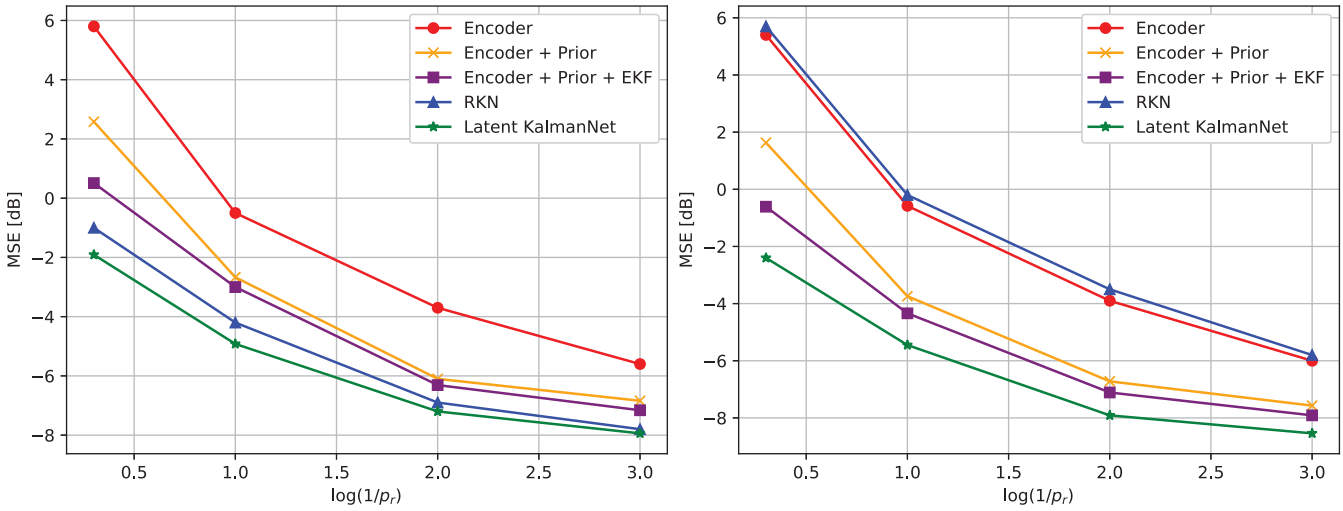
Noise Level $-\log(p_r)$	0.3	1	2	3
Encoder	5.8 $\pm$1.1	-0.5 $\pm$1.3	-3.7 $\pm$0.85	-5.6 $\pm$0.9
Encoder+Prior	2.58 $\pm$0.5	-2.67 $\pm$0.58	-6.1 $\pm$0.48	-6.84 $\pm$0.52
Encoder+Prior+EKF	0.51 $\pm$0.35	-3 $\pm$0.41	-6.31 $\pm$0.38	-7.16 $\pm$0.32
RKN	-1 $\pm$2.1	-4.2 $\pm$1.5	-6.9 $\pm$1.3	-7.8 $\pm$1.1
Latent-KalmanNet	**-1.91 $\pm$0.06**	**-4.92 $\pm$0.1**	**-7.2 $\pm$0.07**	**-7.94 $\pm$0.12**

2) *Encoder + Prior:* The convolutional encoder with a prior estimate stacked to features provided to the first FC layer.
3) *Encoder + Prior + EKF:* Model-based tracking in latent space by applying an EKF with the measurement function set to the identity matrix, i.e., $\mathbf{h}(\mathbf{x}) = \mathbf{x}$. The variance of the state evolution noise is selected through a greed search, while the observation noise is determined by the empirical MSE at the output of the encoder.
4) *RKN:* The RKN of [20], which is a leading data-driven tracking algorithm that utilizes a high-dimensional factorized latent state representation.
5) *Latent-KalmanNet:* The proposed Latent-KalmanNet implemented using the above Encoder Prior along with KalmanNet based on Architecture 2 of [31].

3) Results: To assess Latent-KalmanNet in terms of tracking performance, latency, and robustness, we evaluate MSE as well as latency and complexity. As the S&P noise is characterized by the probability p_r, when evaluating how performance scales with the noise level we report the MSE values versus $\log \frac{1}{p_r}$ for ease of visualization, where $p_r \in \{0.5, \ldots, 0.001\}$ is mapped into $\log \frac{1}{p_r} \in \{0.3, \ldots, 3\}$. We consider both the case of *full information*, where the SS model parameter (e.g., the state evolution function $\mathbf{f}(\cdot)$) is the same as that used for generating the trajectories, and the case of *partial information*, where this domain knowledge is mismatched.

Full Information: Here, we compare Latent-KalmanNet to the benchmark algorithms, where all algorithms have access to the state-evolution function $\mathbf{f}(\cdot)$ used during data generation.

In our first experiment, the trajectory length presented during training was the same as in the test set $T_{\text{train}} = T_{\text{test}} = 200$. The resulting MSEs, reported in Fig. 10(a), demonstrate that the proposed Latent-KalmanNet achieves the lowest MSE for all considered noise levels. The improvement due to adding



(a) Same trajectories length: $T_{\text{train}} = T_{\text{test}} = 200$.

(b) Different trajectories length: $T_{\text{train}} = 200, T_{\text{test}} = 2000$.

Fig. 10. Lorenz system: performance with full domain knowledge. MSE vs. observations S&P noise level.

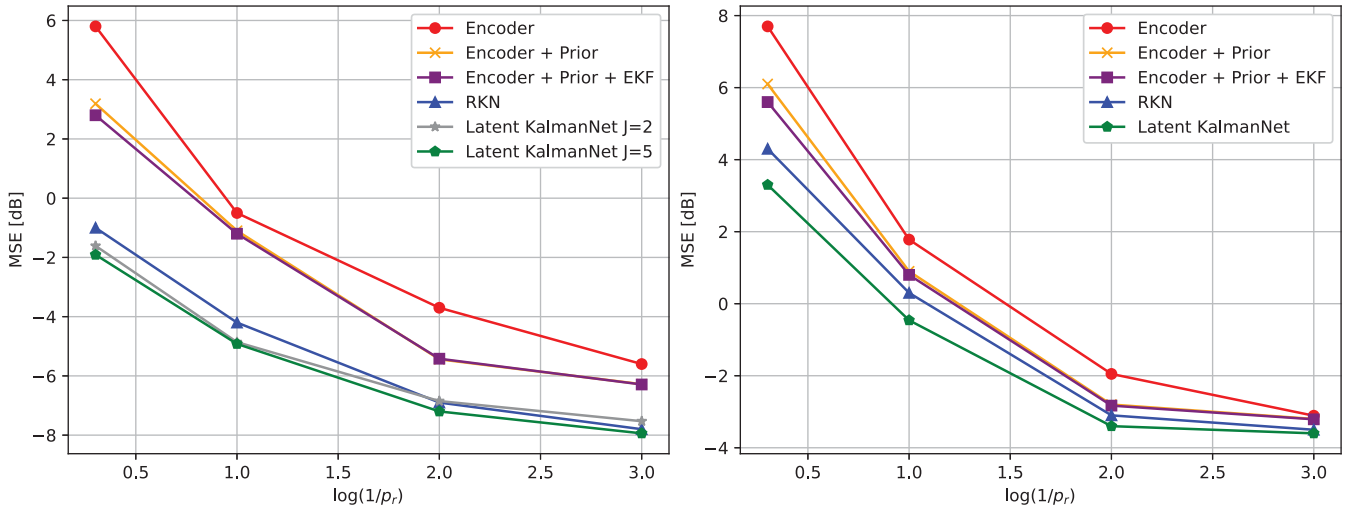
EKF on top of the pre-trained encoder, tracking in the new learned latent space, is much less notable here compared with the improvement in the Pendulum setup noted in Fig. 6(a). This follows since the S&P noise yields a latent representation in which the distribution of the distortion cannot be faithfully approximated as being Gaussian, while the EKF, as opposed to KalmanNet, is designed for Gaussian SS models. The purely data-driven RKN improves upon the model-based EKF, and is only slightly outperformed by Latent-KalmanNet.

The superiority of Latent-KalmanNet is also evident in Table III, which reports the MSEs along with their standard deviation, representing the confident intervals of the estimators. It is observed that the improved estimates of Latent-KalmanNet are also consistently achieved more confidently, i.e., with smaller standard deviation, compared with the competitor RKN. This behavior is also illustrated when observing a single filter trajectory in Fig. 8, where the original state tracked is the one depicted on the left side, and the different models predictions are on the right.

Next, we examine the generalization of the filters to different trajectory lengths. This is done by training the systems with trajectories of length $T_{\text{train}} = 200$ while testing with notably longer trajectories of length $T_{\text{test}} = 2000$. Fig. 10(b) demonstrates how the purely data-driven RKN struggles to generalize, achieving performance that is similar to the stand-alone encoder. However our Latent-KalmanNet successfully generalizes to a much longer trajectory, as it learns how to track based on both data, latent features, and domain knowledge, allowing it to cope with the SS model and not overfit to the trajectory length.

Partial Information: To evaluate the performance of Latent-KalmanNet under partial model information, we consider two sources of model mismatch in the Lorenz system setup. First, we examine state-evolution mismatch due to the use of a Taylor series approximation of insufficient order. In this study, both Latent-KalmanNet and the benchmark algorithms (*Encoder + Prior* and *Encoder + Prior + EKF*) use a crude approximation of the evolution dynamics obtained by computing (24) with $J = 2$, while the data was generated with an order $J = 5$ Taylor series expansion. We set the trajectory length to $T = 200$ (both T_{train} and T_{test}), and $\Delta_t = 0.02$. The results, depicted in Fig. 11(a), demonstrate that applying a model-based EKF achieves performance, which coincides with that of the *Encoder + Prior*. This stems from the fact that mismatched model resulted in the EKF being unable to incorporate the state evolution to improve tracking, and the grid search for identifying the most suitable noise variance lead to the KG computation such that the estimation relies almost solely on the instantaneous observation. More interestingly, Latent-KalmanNet with partial knowledge (of $J = 2$) learns to overcome this model mismatch and manages to come within a small gap with the performance of Latent-KalmanNet with full knowledge (of $J = 5$), outperforming its benchmark counterparts operating with the same level of partial information and the data-drivenRKN. These findings suggest that Latent-KalmanNet is robust and effective, even when operating under partial information of the dynamics.

Next, we evaluate the performance of Latent-KalmanNet in the presence of sampling mismatch. We generate data from the Lorenz system SS model using a dense sampling rate $\Delta_t = 0.001$, and then sub-sample the corresponding observations by a ratio of $\frac{1}{20}$ to obtain a decimated process with sample spacing of $\Delta_t = 0.02$. This results in a mismatch between the SS model and the discrete-time sequence, as the nonlinearity of the SS model results in a difference in distribution between the decimated data and data generated directly with sampling interval $\Delta_t = 0.02$. Such scenarios correspond to the practical setting of mismatches due to processing of continuous-time signals using discrete-time approximations. For this setting we use the identity mapping for the measurement function $\mathbf{h}(\cdot)$ and set $T = 200$ (both T_{train} and T_{test}). The results, shown in Fig. 11(b), demonstrate that Latent-KalmanNet outperforms both model-based tracking and the data-driven RKN as its combination of learning capabilities, along with the available model



(a) Mismatch due to state-evolution Taylor expansion.

(b) Mismatch due to coarse sampling.

Fig. 11. Lorenz system: performance with partially domain knowledge. MSE vs. observations S&P noise level.

TABLE IV
COMPLEXITY COMPARISON

	Encoder+EKF	RKN	Latent-KalmanNet
Complexity (FP)	15573 + 243	42426	15573 + 2712

dynamic knowledge, allows it overcomes the mismatch induced by representing a continuous-time SS model in discrete-time. As in the case with mismatched state evolution, we again observe that the model mismatches result in the EKF being unable to improve performance, and achieving the same performance as that of instantaneous detection using an Encoder + Prior.

C. Complexity and Latency

We conclude our numerical study by demonstrating that the performance benefits of Latent-KalmanNet, do not come at the cost of increased computational complexity and latency, as is often the case when using deep models. In fact, we show that it can achieve faster inference compared with both model-based EKF in latent space and the data-drivenRKN.

To that aim, we provide an analysis of the complexity and the average inference time of these tracking algorithms computed over a test set. The data is comprised of 100 trajectories, with $T_{\text{test}} = 200$ time steps each, both for the Pendulum and the Lorenz system data-sets. We report the number of floating point operations required by each method, where the number of operations in applying a DNN is given by its number of trainable parameters. The resulting complexity measures are reported in Table IV. These results reveal that Latent-KalmanNet has fewer floating points operations than the RKN, thanks to the compact RNN integrated into the Kalman filtering pipeline, having much smaller number of parameters. On the other hand, the computation of the Kalman Gain over the classical EKF with encoder is smaller.

Nonetheless, in terms of inference time, as can be seen in Fig. 12, Latent-KalmanNet is much faster than both the purely

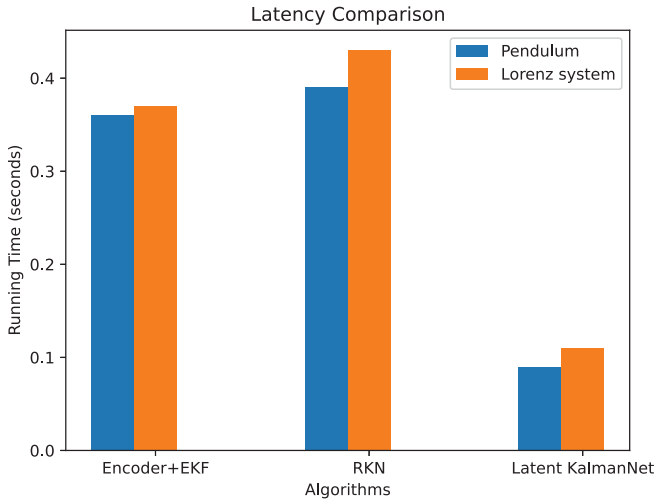


Fig. 12. Running inference time comparison between the filtering methods over both data-sets.

DNN-based RKN and the model-based EKF. The latter stems from the fact that the EKF needs to compute Jacobians and matrix inversions on each time instance to produce its KG, which turns out to be slower compared with applying the compact RNN used by KalmanNet for the same purpose, which is amenable to parallelization and acceleration of built-in software accelerators. Fair comparison is guaranteed by running all methods with standard Python libraries on the same hardware - which is an 11th Gen Lenovo laptop with Intel Core i7, 2.80 GHz processor, 16 GB of RAM, and Windows 11 operating system.

These results of complexity and latency, combined with the performance and robustness gains of Latent-KalmanNet noted in the previous studies, showcase the potential of Latent-KalmanNet in leveraging both data and domain knowledge for tracking with high-dimensional data while coping with the challenging C.1-C.4.

V. CONCLUSION

In this work, we proposed a method for tracking based on complex observations with unknown noise statistics. Our proposed Latent-KalmanNet combines DNN-aided encoding with learned Kalman filtering based on KalmanNet in the latent space, and designs these modules to mutually benefit one another in a synergistic manner. The training scheme of Latent-KalmanNet exploits its interpretable architecture to formulate alternate training between the two learnable components in order to learn a surrogate latent representation, which most facilitates tracking. Our empirical evaluations demonstrate that the proposed Latent-KalmanNet successfully tracks from high-dimensional observations and generalizes to trajectories of different lengths. It also succeeds in working with partial domain knowledge of the state evolution function or sampling mismatches, and is shown to infer with low latency.

ACKNOWLEDGMENT

The authors thank Hans-Andrea Loeliger for the helpful discussions.

REFERENCES

[1] I. Buchnik, D. Steger, G. Revach, R. J. G. van Sloun, T. Routtenberg, and N. Shlezinger, "Learned Kalman filtering in latent space with high-dimensional data," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.

[2] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.

[3] J. Durbin and S. J. Koopman, *Time Series Analysis by State Space Methods*. London, U.K.: Oxford Univ. Press, vol. 38, May 2012.

[4] R. E. Larson, R. M. Dressler, and R. S. Ratner, "Application of the extended Kalman filter to ballistic trajectory estimation," Stanford Res. Inst., Menlo Park, CA, USA, Tech. Rep., 1967. [Online]. Available: https://apps.dtic.mil/sti/citations/AD0815377

[5] E. A. Wan and R. Van Der Merwe, "The unscented Kalman filter," in *Kalman Filtering and Neural Networks*, Hoboken, NJ, USA: Wiley, 2001, pp. 221–280.

[6] P. M. Djuric et al., "Particle filtering," *IEEE Signal Process. Mag.*, vol. 20, no. 5, pp. 19–38, Sep. 2003.

[7] N. J. Gordon, D. J. Salmond, and A. F. Smith, "Novel approach to nonlinear/non-Gaussian Bayesian state estimation," in *Proc. IEE Proc. F (Radar Signal Process.)*, vol. 140, no. 2, Apr. 1993, pp. 107–113.

[8] X. Lin and G. Terejanu, "ENLLVM: Ensemble based nonlinear Bayesian filtering using linear latent variable models," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2019, pp. 5222–5226.

[9] E. Isufi, P. Banelli, P. Di Lorenzo, and G. Leus, "Observing and tracking bandlimited graph processes from sampled measurements," *Signal Process.*, vol. 177, 2020, Art. no. 107749.

[10] G. Sagi and T. Routtenberg, MAP Estimation of Graph Signals, *early access, IEEE Trans. Signal. Process.,* 2022, doi: 10.1109/TSP.2023.3340009.

[11] G. Sagi, N. Shlezinger, and T. Routtenberg, "Extended Kalman filter for graph signals in nonlinear dynamic systems," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.

[12] S. Cheng et al., "Machine learning with data assimilation and uncertainty quantification for dynamical systems: A review," *IEEE/CAA J. Autom. Sin.*, vol. 10, no. 6, pp. 1361–1387, Jun. 2023.

[13] J. Gu, X. Yang, S. De Mello, and J. Kautz, "Dynamic facial analysis: From Bayesian filtering to recurrent neural network," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2017, pp. 1548–1557.

[14] B. Tang and D. S. Matteson, "Probabilistic transformer for time series analysis," in *Proc. Adv. Neural Inf. Process. Syst. 34 (NeurIPS)*, 2021, pp. 23592–23608.

[15] T. Zhi-Xuan, H. Soh, and D. Ong, "Factorized inference in deep Markov models for incomplete multimodal time series," in *Proc. AAAI Conf. Artif. Intell.*. AAAI Press, 2020, vol. 34, no. 6, pp. 10334–10341.

[16] S. S. Rangapuram, M. W. Seeger, J. Gasthaus, L. Stella, Y. Wang, and T. Januschowski, "Deep state space models for time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst. 31 (NeurIPS)*, Montréal, Canada, 2018, pp. 7796–7805.

[17] B. Millidge, A. Tschantz, A. Seth, and C. Buckley, "Neural Kalman filtering," 2021, *arXiv:2102.10021*.

[18] S. Jouaber, S. Bonnabel, S. Velasco-Forero, and M. Pilte, "NNAKF: A neural network adapted Kalman filter for target tracking," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2021, pp. 4075–4079.

[19] D. Ruhe and P. Forré, "Self-supervised inference in state-space models," in *Proc. 10th Int. Conf. Learn. Representations (ICLR)*. OpenReview.net, 2022, pp. 1–21.

[20] P. Becker, H. Pandya, G. Gebhardt, C. Zhao, C. J. Taylor, and G. Neumann, "Recurrent Kalman networks: Factorized inference in high-dimensional deep feature spaces," in *Proc. Int. Conf. Mach. Learn.*, vol. 97, PMLR, 2019, pp. 544–552.

[21] G. Nguyen-Quynh, P. Becker, C. Qiu, M. Rudolph, and G. Neumann, "Switching recurrent Kalman networks," 2021, *arXiv:2111.08291*.

[22] A. Klushyn, R. Kurle, M. Soelch, B. Cseke, and P. van der Smagt, "Latent matters: Learning deep state-space models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 10234–10245.

[23] Y. Wang, X. Luo, L. Ding, and S. Hu, "Visual tracking via robust multi-task multi-feature joint sparse representation," *Multimedia Tools Appl.*, vol. 77, pp. 31447–31467, Jun. 2018.

[24] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation With Applications to Tracking and Navigation: Theory Algorithms and Software*. Hoboken, NJ, USA: Wiley, Jan. 2004.

[25] L. Zhou et al., "KFNet: Learning temporal camera relocalization using Kalman filtering," in *Proc. IEEE/CVF Conf. Comput. Vision Pattern Recognit.*, 2020, pp. 4919–4928.

[26] H. Coskun, F. Achilles, R. DiPietro, N. Navab, and F. Tombari, "Long short-term memory Kalman filters: Recurrent neural estimators for pose regularization," in *Proc. IEEE Int. Conf. Comput. Vision*, 2017, pp. 5524–5532.

[27] M. Fraccaro, S. Kamronn, U. Paquet, and O. Winther, "A disentangled recognition and nonlinear dynamics model for unsupervised learning," in *Proc. Adv. Neural Inf. Process. Syst. 30 (NIPS)*, Long Beach, CA, USA, 2017, vol. 30, pp. 3601–3610.

[28] A. H. Li, P. Wu, and M. Kennedy, "Replay overshooting: Learning stochastic latent dynamics with the extended Kalman filter," in *Proc. IEEE Int. Conf. Robot. Automat (ICRA)*, 2021, pp. 852–858.

[29] R. G. Krishnan, U. Shalit, and D. Sontag, "Deep Kalman filters," 2015, *arXiv:1511.05121*.

[30] B. Laufer-Goldshtein, R. Talmon, and S. Gannot, "A hybrid approach for speaker tracking based on TDOA and data-driven models," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 26, no. 4, pp. 725–735, Apr. 2018.

[31] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. Van Sloun, and Y. C. Eldar, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.

[32] I. Klein, G. Revach, N. Shlezinger, J. E. Mehr, R. J. G. van Sloun, and Y. C. Eldar, "Uncertainty in data-driven Kalman filtering for partially known state-space models," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2022, pp. 3194–3198.

[33] G. Revach, N. Shlezinger, T. Locher, X. Ni, R. J. van Sloun, and Y. C. Eldar, "Unsupervised learned Kalman filtering," in *Proc. Eur. Signal Process. Conf. (EUSIPCO)*, 2022, pp. 1571–1575.

[34] N. Shlezinger, J. Whang, Y. C. Eldar, and A. G. Dimakis, "Model-based deep learning," *Proc. IEEE*, vol. 111, no. 5, pp. 465–499, May 2023.

[35] N. Shlezinger, Y. C. Eldar, and S. P. Boyd, "Model-based deep learning: On the intersection of deep learning and optimization," *IEEE Access*, vol. 10, pp. 115384–115398, 2022.

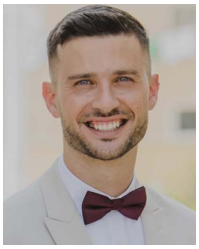
[36] J. Durbin and S. J. Koopman, "A simple and efficient simulation smoother for state space time series analysis," *Biometrika*, vol. 89, no. 3, pp. 603–616, 2002. [Online]. Available: https://apps.dtic.mil/sti/pdfs/AD0654272.pdf

[37] M. Gruber, "An approach to target tracking," MIT Lexington Lincoln Lab, Tech. Rep., 1967.

[38] N. Shlezinger and T. Routtenberg, "Discriminative and generative learning for linear estimation of random signals [lecture notes]," *IEEE Signal Process. Mag.*, vol. 40, no. 6, pp. 75–82, Sep. 2023.

[39] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[40] J. Park and S. Boyd, "General heuristics for nonconvex quadratically constrained quadratic programming," 2017, *arXiv:1703.07870*.
[41] S. Gannot and A. Yeredor, "The Kalman filter," in *Springer Handbook of Speech Processing*, J. Benesty, M. M. Sondhi, and Y.A. Huang, Eds., Berlin, Germany: Springer Handbooks, 2008, pp. 135–160.
[42] D. Bank, N. Koenigstein, and R. Giryes, "Autoencoders," *Machine Learning for Data Science Handbook: Data Mining and Knowledge Discovery Handbook*, L. Rokach, O. Maimon, E. Shmueli (Eds.), Cham, Switzerland: Springer International Publishing, 2023, pp. 353–374.
[43] L. Xu and R. Niu, "EKFNet: Learning system noise statistics from measurement data," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2021, pp. 4560–4564.
[44] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–13.
[45] T. Raviv, S. Park, O. Simeone, Y. C. Eldar, and N. Shlezinger, "Online meta-learning for hybrid model-based deep receivers," *IEEE Trans. Wireless Commun.*, vol. 22, pp. 6415–6431, Oct 2023.
[46] N. Shlezinger and I. V. Bajić, "Collaborative inference for AI-empowered IoT devices," *IEEE Internet Things Mag.*, vol. 5, no. 4, pp. 92–98, Dec. 2022.
[47] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. 30 (NIPS)*, Long Beach, CA, USA, 2017, pp. 5998–6008.
[48] A. Ghosh, A. Honoré, and S. Chatterjee, "DANSE: Data-driven non-linear state estimation of model-free process in unsupervised learning setup," in *Proc. Eur. Signal Process. Conf. (EUSIPCO)*, 2023, pp. 870–874.
[49] J. Lehtinen et al., "Noise2Noise: Learning image restoration without clean data," in *Proc. 35th Int. Conf. Mach. Learn.*, vol. 80, PMLR, 2018, pp. 2971–2980.
[50] K. Zhang, W. Zuo, S. Gu, and L. Zhang, "Learning deep CNN denoiser prior for image restoration," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, 2017, pp. 2808–2817.
[51] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, "A review of YOLO algorithm developments," *Procedia Comput. Sci.*, vol. 199, pp. 1066–1073, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050922001363
[52] W. Gilpin, "Chaos as an interpretable benchmark for forecasting and data-driven modelling," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks 1, NeurIPS Datasets Benchmarks*, 2021, pp. 1–16.
[53] V. G. Satorras, Z. Akata, and M. Welling, "Combining generative and discriminative models for hybrid inference," in *Proc. Adv. Neural Inf. Process. Syst. 32 (NeurIPS)*, Vancouver, BC, Canada, 2019, vol. 32, pp. 13802–13812.
[54] V. Bayle et al., "Single-particle tracking photoactivated localization microscopy of membrane proteins in living plant tissues," *Nature Protocols*, vol. 16, no. 3, pp. 1600–1628, 2021.



Itay Buchnik (Student Member, IEEE) received the B.Sc. and M.Sc. degrees, both *cum laude* from the School of Electrical and Computer Engineering, Ben-Gurion University, Israel, in 2021 and 2023, respectively, specializing in machine learning for signal processing, model-based deep learning, computer vision, and graph signal processing. He was honored with the JNF Scholarship for "Israel 2040," in 2021, the IDSI Grant for "Access and Usage of Cloud Resources," in 2022, and the Kaufman Award for outstanding research achievements, in 2023. He was also the recipient of the Best Student Presentation Award at the 2022 EURASIP Summer School on "Data and Graph Driven Learning for Communications and Signal Processing." During his academic career, he interned with ELTA Systems, the Forschungszentrum Jülich Lab in Germany, and Persimio. He is currently working as an AI Researcher with Elbit Systems.

Guy Revach (Graduate Student Member, IEEE) received the B.Sc. (*cum laude*) and M.Sc. degrees from Andrew and Erna Viterbi Department of Electrical and Computer Engineering, Technion—Israel Institute of Technology, Haifa, in 2008 and 2017, respectively. He completed the master's thesis under the supervision of Prof. Nahum Shimkin on planning for cooperative multiagents. Since 2019, he has been working toward the Ph.D. degree with the Department of Information Technology and Electrical Engineering (D-ITET), Institute for Signal and Information Processing (ISI), ETH Zürich, Switzerland, supervised by Prof. Dr. Hans-Andrea Loeliger. He is a Researcher with a proven industry track record as an Innovator and a System Engineer. His main research focus is on the intersection of machine learning with signal processing, specifically combining sound theoretical principles from classical signal processing with state-of-the-art machine learning algorithms for tracking and detection problems. Before joining ETH Zürich, he worked with the Israeli wireless communication industry for over ten years, initially as a Real-Time Embedded Software Engineer and later as a Software Manager. He was the main Innovator behind state-of-the-art, software-defined radio (SDR) for wireless communication, which was game-changing and groundbreaking in terms of size, weight, and power. As a System Engineer, he defined major aspects of SDR requirements and architecture, including hardware, software, network, cyber defense, signal processing, data analysis, and control algorithms.

Damiano Steger (Student Member, IEEE) received the B.Sc. and M.Sc. degrees from the Department of Information Technology and Electrical Engineering (D-ITET), Institute for Signal and Information Processing (ISI), ETH Zürich, Switzerland, in 2019 and 2022, respectively, specializing in signal processing and machine learning. During his studies, he completed an internship in the financial sector with Credit Suisse, and he is currently employed with UBS.

Ruud J. G. van Sloun (Member, IEEE) received the M.Sc. and Ph.D. degrees (*cum laude*) in electrical engineering from Eindhoven University of Technology, in 2014 and 2018, respectively. Currently, is an Associate Professor with the Department of Electrical Engineering, Eindhoven University of Technology, The Netherlands. From 2019 to 2020, he served as a Visiting Professor with the Department of Mathematics and Computer Science, Weizmann Institute of Science, Rehovot, Israel. From 2020 to 2023, he was a Kickstart AI Fellow with Philips Research. He has been honored with an ERC Starting Grant, an NWO VIDI Grant, an NWO Rubicon Grant, and a Google Faculty Research Award. His current research interests include closed-loop image formation, deep learning for signal processing and imaging, active signal acquisition, model-based deep learning, compressed sensing, ultrasound imaging, and probabilistic signal, and image reconstruction.

Tirza Routtenberg (Senior Member, IEEE) received the B.Sc. degree from Technion Israel Institute of Technology, Haifa, Israel, in 2005, and the M.Sc. and Ph.D. degrees in electrical and computer engineering from Ben-Gurion University, Israel, in 2007 and 2012, respectively. She is an Associate Professor with the School of Electrical and Computer Engineering, Ben-Gurion University, Israel. From 2012 to 2014, she was a Postdoctoral Fellow with the School of Electrical and Computer Engineering, Cornell University. Since October 2014, she has been a Faculty Member with the School of Electrical and Computer Engineering, Ben-Gurion University of the Negev, Beer-Sheva, Israel. From 2022 to 2023, she was a William R. Kenan, Jr., Visiting Professor for Distinguished Teaching with Princeton University. Her research interests include signal processing in smart grids, statistical signal processing, estimation and detection theory, and signal processing on graphs. She is an Associate Editor of IEEE TRANSACTIONS ON SIGNAL AND INFORMATION PROCESSING OVER NETWORKS and IEEE SIGNAL PROCESSING LETTERS, as well as a member of IEEE Signal Processing Theory and Methods Technical Committee. She was a recipient of the Best Student Paper Award at the International Conference on Acoustics, Speech and Signal Processing (ICASSP) 2011, IEEE International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP) 2013 (Co-Author), ICASSP 2017 (Co-Author), and IEEE Workshop on Statistical Signal Processing (SSP) 2018 (Co-Author). She was awarded the Negev scholarship in 2008, the Lev-Zion scholarship in 2010, the Marc Rich Foundation Prize in 2011, and the Toronto Prize for Excellence in Research in 2021.

Nir Shlezinger (Senior Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees from Ben-Gurion University, Israel, all in electrical and computer engineering, in 2011, 2013, and 2017, respectively. Currently, is an Assistant Professor with the School of Electrical and Computer Engineering, Ben-Gurion University, Israel. From 2017 to 2019, he was a Postdoctoral Researcher with Technion, and from 2019 to 2020, he was a Postdoctoral Researcher with Weizmann Institute of Science, where he was awarded the FGS Prize for outstanding research achievements. His research interests include communications, information theory, signal processing, and machine learning.